



Maitsys Assure

Self-assessment and corrective-action platform

Architecture, Information Architecture & User Flow Documentation

Complete technical and experience reference for the current application

Prepared	25 August 2026
Scope	Full application — all three user roles (Site Contributor, Enterprise/Regional Viewer, Administrator)
Stack	React 19 · TypeScript · React Router 7 · Vite 6
Source	kc-ehss-self-assessment (package: maitsys-assure)

Table of Contents

1	Product Overview	
2	System Architecture	
3	Technology Stack	
4	Information Architecture (Site Map)	
5	Roles, Navigation & Access Control	
6	User Flow — Authentication & Onboarding	
7	User Flow — Site Contributor	
8	User Flow — Enterprise / Regional Viewer	
9	User Flow — Administrator	
10	Data Model & Entity Relationships	
11	State Management & Data Flow	
12	Feature Module Reference	
13	Governance, Audit & History	
14	Non-Functional Notes & Architecture Rules	

Product Overview

Maitsys Assure is a role-aware EHS&S (Environment, Health, Safety & Sustainability) self-assessment and corrective-action platform. Sites complete a governed self-assessment against Operating System and Performance Standard requirements; enterprise leadership monitors completion and performance across their authorized scope; administrators govern the master requirement catalog, control who is assigned where, and audit every change.

Who uses it

ROLE	HOME SCREEN	PRIMARY PURPOSE
Site Contributor	/overview	Owns exactly one assigned site. Maintains site/regional contacts and program owners, answers every assessment question, attaches evidence, and closes corrective-action gaps.
Enterprise / Regional Viewer	/dashboard	Read-only oversight across an authorized scope of sites. Filters, compares completion & performance, drills into any site or section, exports reporting views.
Administrator	/dashboard	Owns the governed master requirement catalog (create, edit, version, publish, import), manages the site network and site user assignments, and reviews the full audit trail of governed changes.

Core product loop

An administrator authors or imports **Master Requirements** (the governed catalog) → publishing reconciles them into every scoped site's live **Assessment** → a Site Contributor answers each question, attaches evidence, and records corrective actions for gaps → completion and performance roll up in real time to the Enterprise Dashboard → every governed change and every response change is preserved in an append-only history / audit trail.

Every response follows a fixed three-level scale: **No** → **Initial**, **Partial** → **Emerging**, **Yes** → **Performing**. A requirement's overall result is always its *lowest* question result — one weak answer holds the whole requirement back, by design.

System Architecture

The application is a single-page React app organized into five strict layers, enforced automatically by a custom architecture linter (`tools/check-architecture.mjs` , run on every `npm run verify`). No layer may reach past the one directly below it.

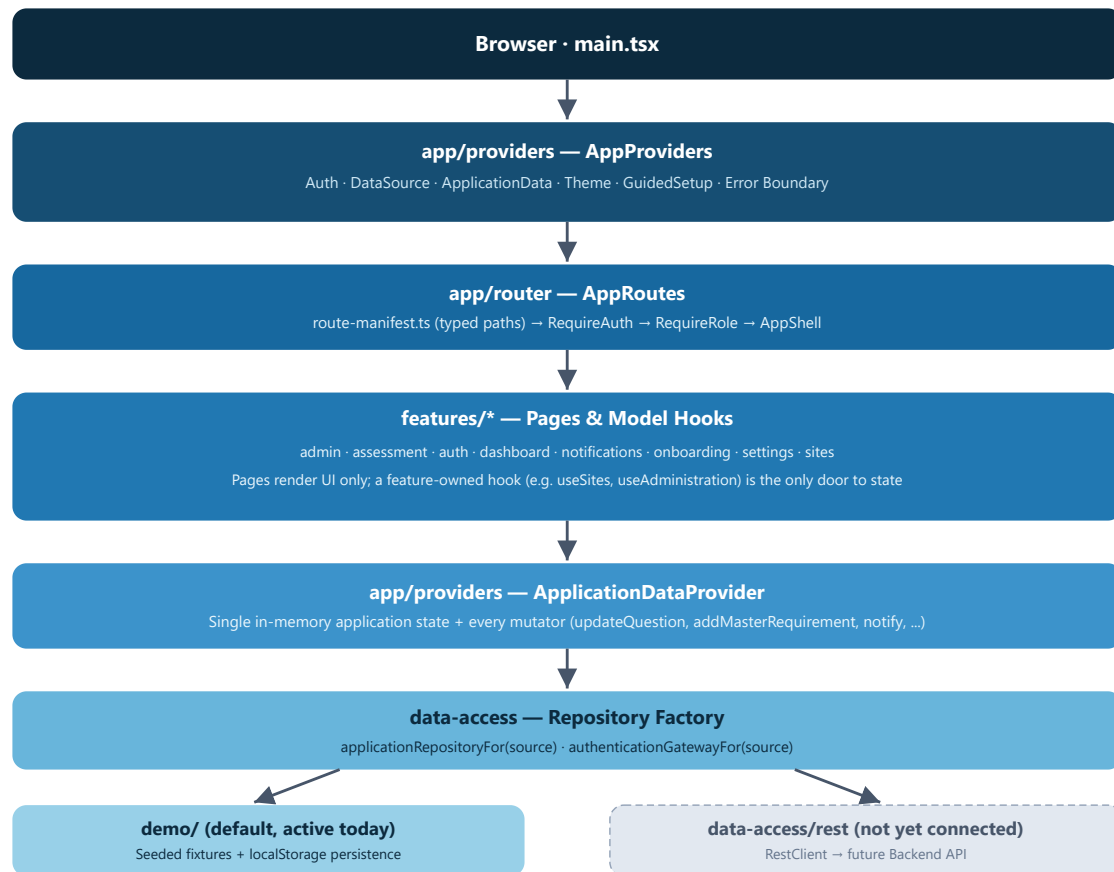


Fig. 1 — Layered runtime architecture. UI never touches fetch, demo fixtures, or REST transport directly.

Why this layering exists

- **Pages never import fixtures or transport.** A page only ever calls its feature's hook (e.g. `useDashboard()`); the hook is the only door to `ApplicationDataProvider` .
- **One state model, one source of truth.** All business state (assessments, sites, users, imports, notifications, audit log) lives in a single `ApplicationDataProvider` — there is no second cache to fall out of sync.
- **Demo and API are interchangeable at one seam.** `applicationRepositoryFor(source)` is the only place that decides whether state comes from seeded fixtures or a real backend. Switching sources reloads the whole app so the two can never mix mid-session.
- **Enforced, not just documented.** The architecture rules above are checked by a script on every build — a page importing REST details, a raw `fetch()` outside the REST layer, or a demo import outside the repository layer fails `npm run verify` .

Technology Stack

Runtime dependencies

PACKAGE	VERSION	ROLE
react / react-dom	^19.0.0	UI runtime
react-router-dom	^7.1.1	Routing, guards, nested layouts
lucide-react	^0.468.0	Icon set used throughout the UI

Build & quality tooling

TOOL	PURPOSE
Vite 6	Dev server + production bundling
TypeScript ~5.7 (strict)	Static typing across the whole codebase
ESLint 9 + typescript-eslint	Lint rules, React Hooks rules
Custom architecture linter	Enforces the layering in Section 2

Styling is a single hand-written stylesheet (`src/styles.css`) — no CSS framework or component library dependency. There is currently no automated test runner (no Jest/Vitest); quality is enforced through `npm run verify` (lint → typecheck → architecture check → production build).

npm scripts

dev

build

lint

typecheck

check:architecture

verify

preview

Information Architecture (Site Map)

28 registered routes, defined once in `route-manifest.ts` and never hard-coded elsewhere. Every route declares its own access rule (public, any authenticated user, or a specific role set).

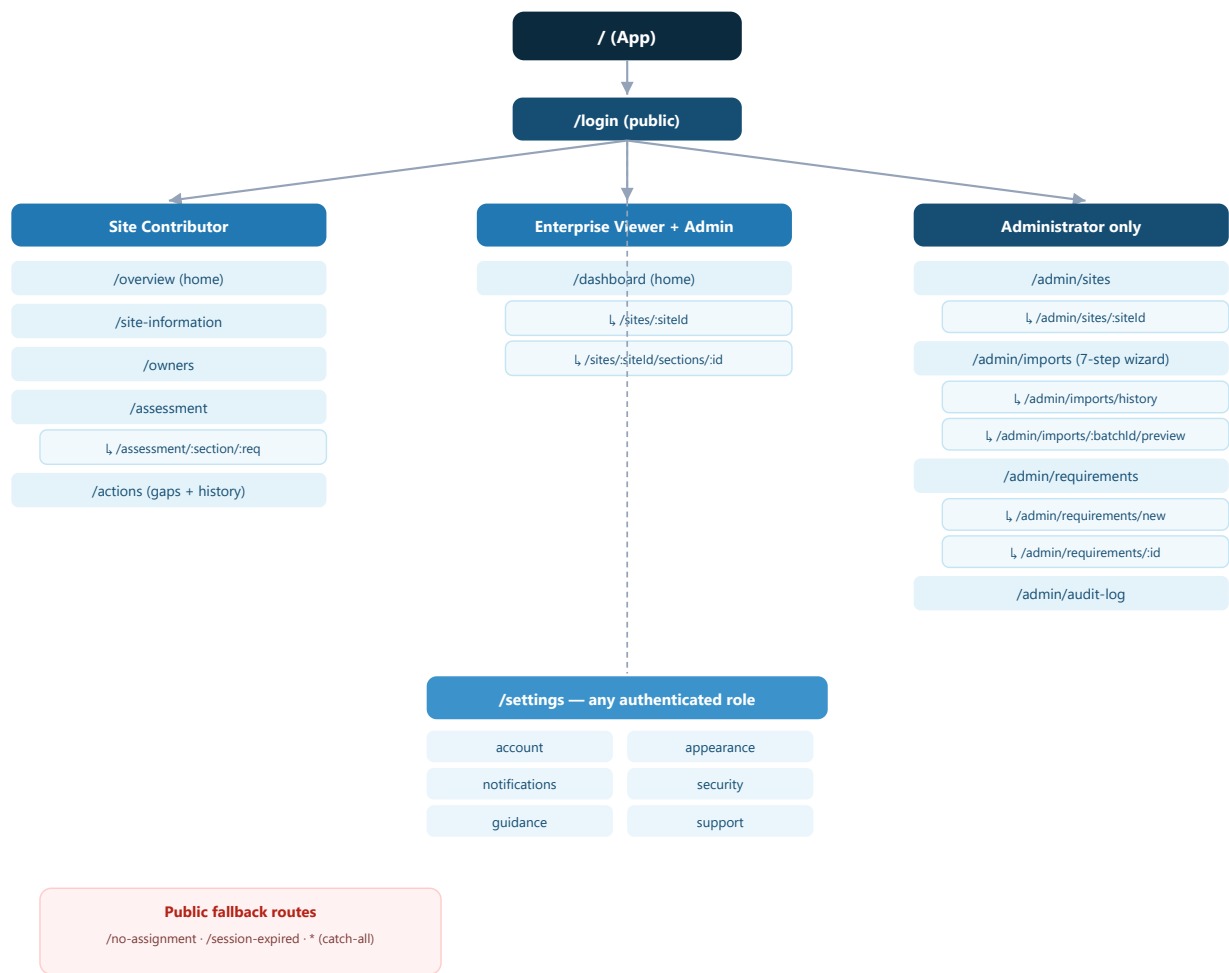


Fig. 2 — Full site map grouped by role. Nested rows are non-navigation detail routes reached from a parent screen.

Roles, Navigation & Access Control

Side navigation, by role (desktop)

GROUP	SITE CONTRIBUTOR	ENTERPRISE / REGIONAL VIEWER	ADMINISTRATOR
Site workspace	Overview · Site information · Program owners · Self-assessment · Actions summary	—	—
Oversight	—	Enterprise dashboard	Enterprise dashboard
Administration	—	—	Sites · Imports · Master requirements · Audit log

Mobile uses a bottom tab bar (role-filtered, 3–4 primary tabs) plus a "More" drawer holding the same full navigation set. A collapsible desktop sidebar state persists per browser.

Access control model

- **RequireAuth** — redirects an anonymous visitor to `/login`, remembering the page they were trying to reach so sign-in returns them there.
- **RequireRole** — redirects a signed-in user whose role isn't allowed on a route to *their own* role home (Site Contributor and Enterprise Viewer/Administrator can never land on each other's screens, even by URL).
- Guards are a **navigation-only** boundary. The architecture explicitly notes that the real backend, once connected, must independently re-enforce identity, scope, and resource ownership — the frontend guard is not a security control on its own.

Scope model

Site Contributor is pinned to exactly one *assigned site* — every editable screen operates on that one site only, and the site badge in the sidebar cannot be changed by the user.

Enterprise/Regional Viewer & Administrator operate over an *authorized scope* of sites (filterable), and every record outside their own edit surface (e.g. another site's contacts) renders strictly read-only.

User Flow — Authentication & Onboarding

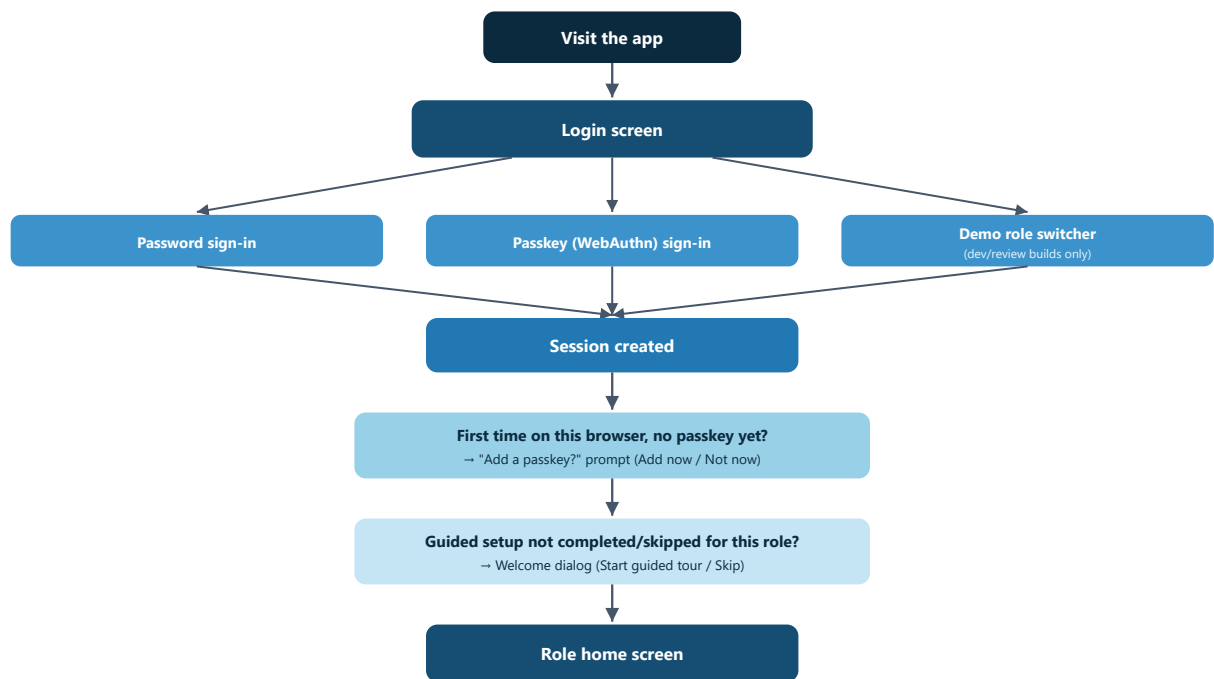


Fig. 3 — First-login sequence: session → optional passkey prompt → optional guided-setup welcome → role home.

Guided setup tours (per role)

SITE CONTRIBUTOR (6 STEPS)	ENTERPRISE VIEWER (4 STEPS)	ADMINISTRATOR (6 STEPS)
1. Confirm your assigned site 2. Resume the next incomplete requirement 3. Maintain site & regional contacts 4. Confirm Primary/Backup owners 5. Work through the self-assessment 6. Track every assessment gap	1. Focus the enterprise dashboard 2. Compare authorized sites 3. Export the current filtered scope 4. Review a site's section-level detail	1. Monitor enterprise adoption 2. Select the approved import workbook 3. Inspect / map / validate / confirm 4. Trace every completed import 5. Find master requirements 6. Create governed requirements

A spotlight coachmark walks each step against the live UI; skipping surfaces a persistent "Continue setup" reminder banner until finished, dismissed, or replayed later from Settings → Guidance.

Session edge cases

- **No assignment** (`/no-assignment`) — reached when sign-in succeeds but the account has no role/site assignment; offers Contact support or Sign out.
- **Session expired** (`/session-expired`) — signs the user out immediately, warns that unsaved (unconfirmed) work may be lost, and offers to sign in again.

SECTION 7

User Flow — Site Contributor

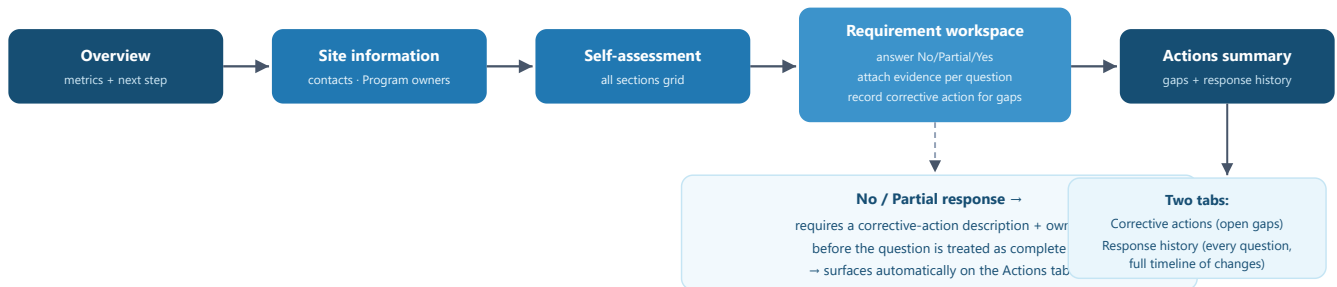


Fig. 4 — The site contributor's assessment loop. "Next incomplete requirement" shortcuts appear on Overview and the Assessment home to keep this loop fast.

Requirement workspace detail

- Left navigator: search across all requirements, overall progress, one-click "Next incomplete."
- Right guidance panel ("How to meet" + "Expected evidence") — read-only governed content, collapsible/minimized by default to keep focus on the questions.
- Each question: response selector → conditional corrective-action editor (only appears/required for No or Partial) → per-question evidence attachments (file upload or secure link).
- Previous/Next requirement footer keeps the whole assessment navigable without returning to the grid.

User Flow — Enterprise / Regional Viewer



Fig. 5 — Enterprise/Regional Viewer journey: filter → drill down → question-level detail, never leaving read-only territory.

SECTION 9

User Flow — Administrator

9.1 — Governed data import



Fig. 6 — Governed workbook import wizard. Imported rows land as Draft and are invisible to sites until explicitly published.

9.2 — Master requirement governance & audit

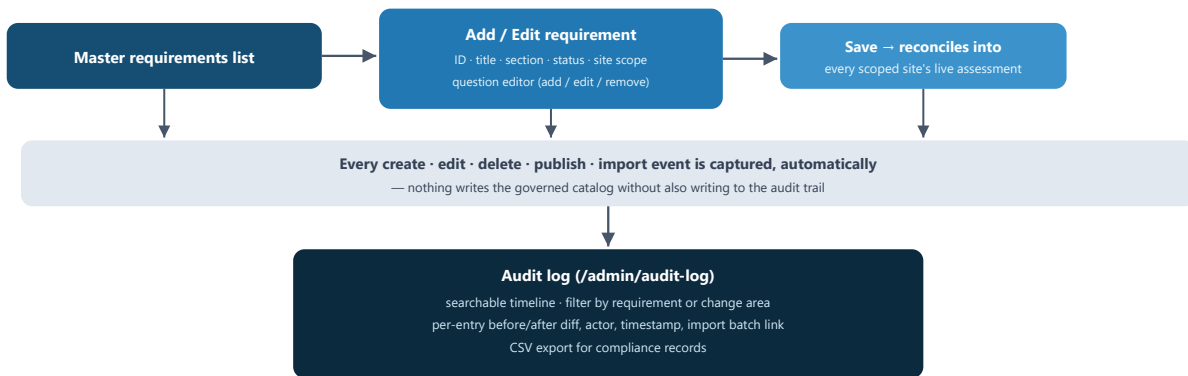


Fig. 7 — Master requirement authoring flow and its always-on audit trail.

Additional administrator surfaces

- **Sites** (</admin/sites>) — create/edit sites individually or via CSV bulk import; per-site detail page manages assigned users and shows read-only owners/contacts.
- The question editor supports marking evidence as required per question and attaching a list of expected evidence items, matching what a Site Contributor is asked for on that exact question.

Data Model & Entity Relationships

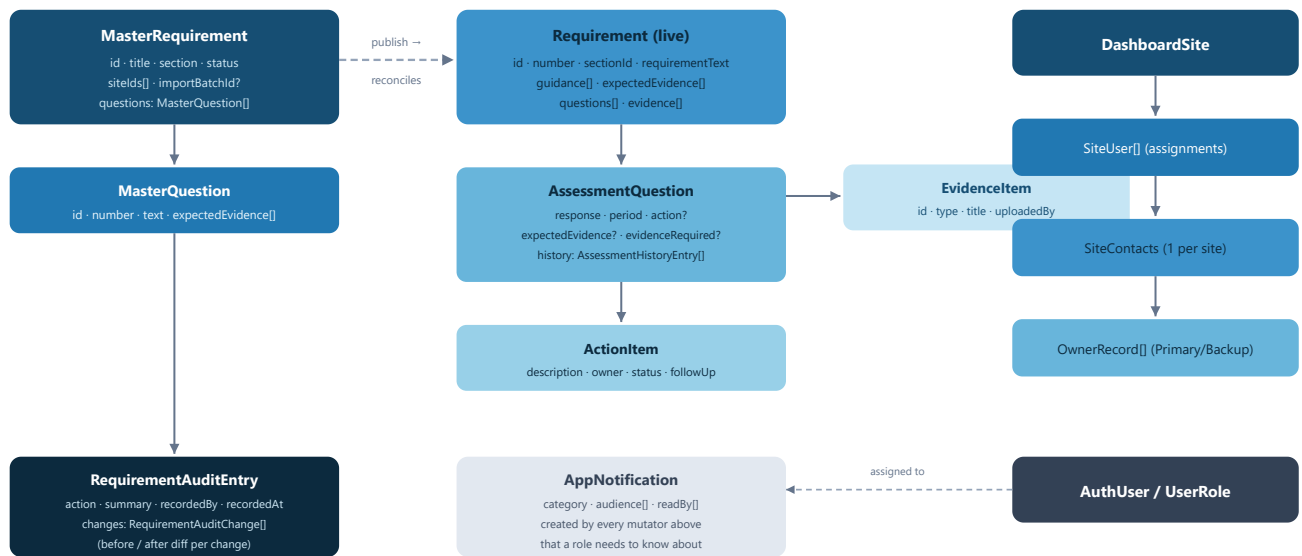


Fig. 8 — Simplified entity relationships. The governed catalog (left) and the live per-site assessment (center) are separate, reconciled models — see Section 11.

Why two requirement models exist

The application deliberately separates **governed definition** from **live answer state**:

- **MasterRequirement / MasterQuestion** — the administrator-owned catalog: what a requirement *is*, what evidence it expects, which sites it applies to. This is what gets imported, versioned, and published.
- **Requirement / AssessmentQuestion (live)** — what a specific site actually sees and answers: the same question text and evidence expectation, plus the site's *own* response, corrective action, evidence attachments, and history.

Publishing a Master Requirement *reconciles* its questions into every scoped site's live copy — new questions are added unanswered, edited text/evidence updates in place, and removed questions are deleted — while every site's own recorded answers, actions, and evidence for questions that still exist are always preserved untouched.

SECTION 11

State Management & Data Flow

All business state lives in one provider — `ApplicationDataProvider` — backed by a repository selected once at startup. There is intentionally no second server-state cache (see ADR-0001); adding one before a real backend exists would just duplicate the same local fixtures under a different name.

Full mutator surface (every way state can change)

MUTATOR	EFFECT
<code>updateQuestion</code>	Sets a response/action/period; auto-creates a corrective action for No/Partial; appends history
<code>add / update / removeEvidence</code>	Manage evidence items on a question; each appends history
<code>saveSiteContacts</code>	Replaces the contacts record for the assigned site
<code>updateOwner</code>	Updates one Program/Standard owner record
<code>add / update / removeMasterRequirement</code>	Governs the catalog; reconciles into live requirements; writes an audit entry
<code>submitImportBatch / publishImportBatch</code>	Stages then publishes a governed workbook import; writes import history + audit entries
<code>add / update / removeSiteUser</code>	Manages who is assigned to a site, in what role
<code>add / updateSite / importSites</code>	Manages the site network, individually or via CSV
<code>notify / markNotificationRead(All)</code>	Creates and reads role-scoped notifications

Demo ↔ live backend seam

Every one of the mutators above runs against whichever repository `applicationRepositoryFor(source)` returns. Today, `source` is always `"demo"` — a seeded fixture set persisted to `localStorage`. An `"api"` mode already exists in the contracts layer for a future backend; its repository currently returns an empty snapshot rather than silently falling back to demo data. Cutting over means implementing the REST resources documented in `docs/api-integration.md` — no feature page changes.

Required REST resources (documented contract, not yet implemented)

AREA	ENDPOINTS
Session	<code>POST/DELETE /auth/session</code>
WebAuthn	<code>/auth/passkeys/registration authentication/options verify</code>
Assessment	<code>GET /sites/:id/assessment</code> , <code>PATCH /requirements/:id/questions/:id</code>
Evidence	<code>POST/PATCH/DELETE /requirements/:id/evidence[:id]</code> (multipart)

Sites	<code>GET /sites[:id]</code> , <code>PUT /sites/:id/contacts</code> , <code>GET /sites/:id/owners</code>
Administration	site users CRUD, <code>POST /imports/requirements</code> , <code>POST /imports/:id/publish</code>
Notifications	<code>GET /notifications</code> , mark-read endpoints
Preferences	<code>GET/PUT /preferences</code> (optional future sync)

Feature Module Reference

MODULE	SCREENS	MODEL HOOK
admin	Sites list · Site detail · Import wizard · Import history · Import preview · Master requirements list · Requirement editor · Audit log	<code>useAdministration()</code>
assessment	Requirement workspace	<code>useAssessment()</code>
auth	Login · No-assignment · Session-expired · First-login passkey prompt	<code>useAuth()</code>
dashboard	Enterprise dashboard · Site drill-down · Section detail	<code>useDashboard()</code>
notifications	Bell-icon popover (no standalone screen)	<code>useNotifications(role)</code>
onboarding	Welcome / Coachmark / Help / Completion dialogs, setup reminder banner	<code>useGuidedSetup()</code>
settings	Account · Appearance · Notifications · Security · Guidance · Support	<code>useTheme()</code> , <code>useAuth()</code> , <code>useGuidedSetup()</code>
sites	Overview · Assessment home · Site information · Owners · Actions summary	<code>useSites()</code>

Every feature exposes exactly one public `index.ts` ; nothing outside a feature imports its internal files directly (enforced by the architecture linter).

Governance, Audit & History

Requirement Audit Log

Every governed-catalog write — created, updated, deleted, imported, published — is recorded to an append-only `RequirementAuditEntry` list, independent of the record it describes (a deleted requirement's history remains visible, labeled accordingly). Each entry captures the actor, timestamp, a human-readable summary, and a structured before/after diff per changed field (requirement text, publishing state, site scope, individual questions, or expected evidence). Administrators can search, filter by requirement or change area, and export the full log to CSV for compliance records.

Response History

Every individual question keeps its own append-only `AssessmentHistoryEntry[]` — response recorded, response changed, action added/updated/removed, evidence added/updated/removed — each capturing the actor, timestamp, and a snapshot of the response/action/evidence-count at that moment. This surfaces in two places:

- **Site Contributor** → **Actions** → **Response history tab** — every question (not only gaps), filterable by response and assessment period.
- **Enterprise/Admin** → **Site drill-down** → **Section detail** — the same timeline, read-only, per question.

Collapsible Guidance

The requirement workspace's governed "How to meet" / "Expected evidence" panel is minimized by default so it never competes with the questions themselves, and can be restored with one click (or opened as a slide-in sheet on mobile). This is a distinct interaction from the app shell's own collapsible left navigation.

Non-Functional Notes & Architecture Rules

Enforced automatically on every `npm run verify`

- No application code may live directly at the `src` root.
- `fetch()` is only permitted inside `src/data-access/rest`.
- Raw `import.meta.env` access is only permitted inside the environment config module.
- `src/demo` may only be imported by the data-access repository composition layer — never by a feature or page directly.
- Pages may not import REST transport details.
- Every feature folder must expose a public `index.ts`.
- Route paths must come from `route-manifest.ts` — no literal path strings in `<Route>` JSX, and no duplicate route-manifest entries.

Removing the demo layer, when a real backend exists

1. Implement the documented endpoints in `src/data-access/rest/repositories.ts`.
2. Replace the empty-snapshot API repository with real, asynchronous resource loading.
3. Set `VITE_API_BASE_URL` and disable `VITE_ENABLE_DEMO_AUTH`.
4. Remove the development-only data-source switch.
5. Delete `src/demo` — the architecture check will surface any remaining references.

Behavior-preservation contract

Architectural refactors are explicitly required to never change route URLs, redirect outcomes, role visibility/navigation, rendered copy or CSS class names, demo credentials/seeded data, local-storage keys, or any of the guided-setup, theme, evidence, import, notification, or passkey flows — any intentional product change is treated as a separate, reviewed task.

Generated from the current application source (routes, providers, feature modules, and data model) on 25 August 2026. This document reflects the live codebase at the time of generation; regenerate after material architecture or navigation changes.